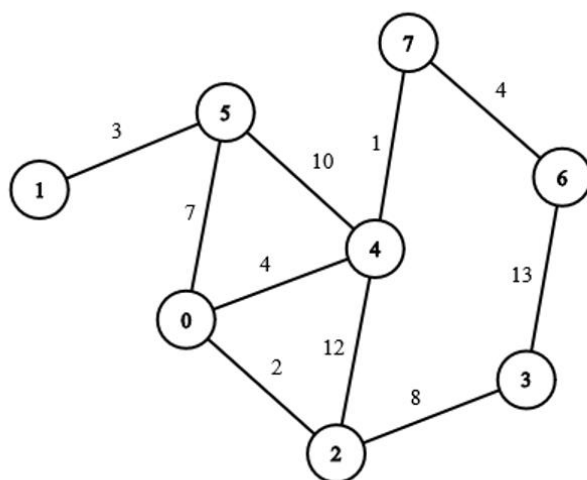


Atestat la informatică

Algoritmi pentru Arborele Parțial de cost Minim



Profesor îndrumător
Diaconescu Adriana

Autor
Pop Tudor

Baia Mare
2026

Contents

I. Introducere / Argument	1
II. Prezentarea mediului utilizat în realizarea aplicației	2
III. Prezentarea aplicației	3
IV. Utilizarea aplicației	5
V. Cod sursă	7
Bibliografie	11

I. Introducere / Argument

Lucrarea de față prezintă doi algoritmi clasici pentru determinarea Arborelui Parțial de cost Minim (APM) într-un graf neorientat ponderat. Arborele Parțial de cost Minim (APM) reprezintă un subgraf conex al unui graf neorientat ponderat, care include toate nodurile grafului inițial, nu conține cicluri și are proprietatea că suma costurilor muchiilor componente este minimă posibilă. Cu alte cuvinte, dintr-un graf cu n noduri, APM-ul va conține întotdeauna exact $n-1$ muchii, conectând toate nodurile cu costul total cel mai mic.

Această structură de date fundamentală din teoria grafurilor își găsește aplicabilitatea într-o gamă largă de domenii reale. În **telecomunicații și rețele de calculatoare**, APM-ul este folosit pentru proiectarea rețelelor locale (LAN) sau a infrastructurilor de fibră optică, minimizând lungimea totală a cablurilor necesare pentru conectarea tuturor nodurilor. În **inginerie civilă**, algoritmi de tip APM stau la baza planificării rețelelor de drumuri, conducte de gaz sau sisteme de irigații, unde obiectivul este conectarea tuturor punctelor cu un cost de construcție minim. În domeniul **Machine Learning**, APM-ul este utilizat în algoritmi de clustering (precum Single-Linkage Clustering), ajutând la identificarea grupurilor naturale din seturi mari de date. Nu în ultimul rând, în **protocoalele de rutare din rețele**, algoritmi precum OSPF (Open Shortest Path First) se bazează pe concepte similare pentru a determina căile optime de transmitere a datelor.

Datorită acestor aplicații diverse și importanței sale teoretice, problema determinării APM-ului a generat mai mulți algoritmi eficienți, dintre care cei mai cunoscuți sunt algoritmul lui Kruskal și algoritmul lui Prim, ambii de tip greedy, prezentați în această lucrare.

Am ales această temă deoarece algoritmi greedy reprezintă una dintre cele mai elegante metode de rezolvare a problemelor de optimizare în teoria grafurilor. Aplicația permite utilizatorului să introducă un graf neorientat ponderat și să vizualizeze rezultatele ambilor algoritmi, facilitând înțelegerea modului în care aceștia funcționează.

II. Prezentarea mediului utilizat în realizarea aplicației

Aplicația a fost realizată în limbajul C++, folosind mediul de dezvoltare **Visual Studio Code** (VS Code) 1.89, un editor de cod sursă gratuit și open-source, dezvoltat de Microsoft, disponibil pe platformele Windows, Linux și macOS.

Caracteristici principale ale Visual Studio Code:

- Editor de cod cu evidențierea sintaxei (syntax highlighting) și autocompletare inteligentă (IntelliSense);
- Suport nativ pentru Git și controlul versiunilor;
- Terminal integrat pentru compilare și rulare directă din editor;
- Sistem extensibil de plugin-uri (extensii);
- Complet gratuit, fără licență comercială.



Pentru compilarea și rularea programului C++, s-a utilizat extensia **CodeLLDB**, un debugger nativ bazat pe LLDB, care permite compilarea și rularea codului C++ direct din VS Code, fără a fi necesară utilizarea manuală a terminalului. Extensia oferă suport pentru breakpoint-uri, inspecția variabilelor în timp real și urmărirea execuției pas cu pas, facilitând astfel depistarea și corectarea erorilor din cod.

Codul sursă a fost scris respectând standardul C++11, utilizând doar tipuri de date de bază, tablouri statice și structuri simple, fără a folosi biblioteca STL (vector, list etc.), pentru a demonstra înțelegerea aprofundată a algoritmilor.

Față de alternative precum **Code::Blocks** sau **Dev-C++**, VS Code oferă o experiență modernă, cu suport pentru extensii, teme vizuale și integrare Git, fiind totodată mai ușor de configurat pentru proiecte de dimensiuni mici precum această aplicație.

III. Prezentarea aplicației

Aplicația implementează doi algoritmi clasici pentru APM, prezentați în cadrul unui meniu interactiv în consolă.

Algoritmul lui Kruskal:

Kruskal este un algoritm de tip greedy care construiește APM-ul prin selectarea succesivă a muchiilor de cost minim, evitând formarea ciclurilor. Se folosește tehnica **Union-Find** implementată cu un tablou de reprezentanți.

Pașii algoritmului Kruskal:

1. Se citește graful (n noduri, m muchii cu costuri).
2. Se sortează muchiile crescător după cost (BubbleSort).
3. Se inițializează fiecare nod ca propriul reprezentant.
4. Se parcurg muchiile sortate: **dacă extremitățile au reprezentanți diferiți**, muchia se adaugă în APM și cei doi subarbori se reunesc.
5. Se repetă până la obținerea **n-1 muchii**.

Algoritmul lui Prim:

Prim construiește APM-ul pornind dintr-un nod sursă, extinzând progresiv mulțimea nodurilor din APM prin adăugarea muchiei de cost minim care conectează un nod din APM cu unul din exterior. Se folosește matricea costurilor și tablouri auxiliare.

Pașii algoritmului Prim:

1. Se citește graful (matricea costurilor $\text{cost}[i][j]$).
2. Se inițializează $\text{minCost}[i] = \text{infinit}$ pentru toți, $\text{minCost}[1] = 0$.
3. La fiecare pas se alege **nodul nevizitat u cu $\text{minCost}[u]$ minim**.
4. Se marchează u ca fiind în APM și se actualizează costurile vecinilor săi.
5. Se repetă de n ori.

Analiza comparativă a algoritmilor:

Criteriu	KRUSKAL	PRIM
Complexitate timp	$O(m \log m)$	$O(n^2)$
Structură internă	Union-Find + BubbleSort	Matrice de costuri
Potrivit pentru	Grafuri rare (m mic)	Grafuri dense
Pornire	Din muchia de cost minim	Dintr-un nod ales

Când se folosește Kruskal vs Prim:

Kruskal este mai eficient pe grafuri **rare** (puține muchii față de noduri), deoarece sortează și parcurge doar muchiile existente. De exemplu, o rețea de drumuri între 100 de orașe cu 150 conexiuni.

Prim este preferat pe grafuri **dense** (multe muchii), deoarece complexitatea $O(n^2)$ depinde doar de numărul de noduri, nu de muchii. De exemplu, o rețea completă unde fiecare oraș e conectat cu toate celelalte.

Limitări ale implementării actuale și posibile îmbunătățiri:

Sortarea **Bubble Sort** din Kruskal are complexitate $O(m^2)$. O îmbunătățire ar fi utilizarea algoritmului QuickSort ($O(m \log m)$), care ar face implementarea mai eficientă pe grafuri mari.

Algoritmul Prim în varianta cu matrice are complexitatea $O(n^2)$. Utilizând o **coadă de priorități** (heap binar), aceasta ar putea fi redusă la $O(m \log n)$, dar implică structuri de date mai avansate.

Implementarea actuală suportă:

- Maximum **100 de noduri** (limita tabloului cost[101][101] și a vectorilor auxiliari)
- Maximum **5000 de muchii** (limita tabloului de structuri x[5000])
- Costurile muchiilor trebuie să fie **numere întregi pozitive**, mai mici decât 999999 (valoarea folosită ca infinit în algoritmul Prim)
- Nodurile trebuie numerotate de la **1 la n**, consecutiv

Aceste limite sunt suficiente pentru scopuri educaționale și pentru testarea manuală a algoritmilor, însă pentru grafuri foarte mari ar fi necesară o implementare cu structuri dinamice și algoritmi de sortare mai eficienți.

IV. Utilizarea aplicației

La pornire, aplicația afișează un meniu interactiv în consolă, din care utilizatorul poate alege algoritmul dorit.

Utilizatorul introduce 1 sau 2 pentru a selecta algoritmul. Orice altă valoare afișează mesajul „Opțiune invalidă!” și meniul se reia.

```
=====
      ALGORITMI - ARBORE PARTIAL MINIM
=====
1. Algoritmul lui Kruskal
2. Algoritmul lui Prim
-----
Alegeti algoritmul:
```

Utilizarea algoritmului Kruskal:

După selectarea opțiunii 1, utilizatorul este rugat să introducă numărul de noduri și numărul de muchii, urmat de datele fiecărei muchii (nodul de start, nodul de sfârșit și costul acesteia):

```
[Kruskal]
Introduceti numarul de noduri si numarul de muchii: 6 9
Introduceti muchiile (nod1 nod2 cost):
Muchia 1: 1 2 4
Muchia 2: 1 3 2
Muchia 3: 2 4 5
Muchia 4: 2 3 1
Muchia 5: 3 4 8
Muchia 6: 3 5 10
Muchia 7: 4 6 2
Muchia 8: 5 6 3
Muchia 9: 2 5 7
```

Aplicația sortează intern muchiile după cost și aplică algoritmul, afișând apoi costul total al APM-ului și muchiile selectate:

```
Cost APM (Kruskal): 13
Muchiile din APM:
2 -- 3 (cost 1)
1 -- 3 (cost 2)
4 -- 6 (cost 2)
5 -- 6 (cost 3)
2 -- 4 (cost 5)
```

Utilizarea algoritmului Prim:

După selectarea opțiunii 2, introducerea datelor se face în același format ca la Kruskal. Algoritmul pornește din nodul 1 și extinde progresiv APM-ul, afișând la final același tip de rezultat:

```
Cost APM (Prim): 13
Muchiile din APM:
 3 -- 2 (cost 1)
 1 -- 3 (cost 2)
 2 -- 4 (cost 5)
 6 -- 5 (cost 3)
 4 -- 6 (cost 2)
```

Se poate observa că ambii algoritmi produc același cost total (19), însă ordinea muchiilor în output poate diferi, deoarece există mai mulți arbori parțiali minimi posibili pentru același graf.

Continuarea sau închiderea aplicației:

După afișarea rezultatelor, utilizatorul este întrebat dacă dorește să continue:

```
Doriti sa continuati? (1 = Da / 0 = Nu):
```

Introducând 1, meniul principal se reia și se poate testa un alt graf sau alt algoritm. Introducând 0, aplicația afișează mesajul „La revedere!” și se închide.

Un lucru important de știut la testele mari:

Pot exista **mai multe APM-uri valide** cu același cost, deci dacă muchiile afișate diferă, nu înseamnă că rezultatul este greșit. Costul total trebuie să fie **același!**

V. Cod sursă

Structuri de date utilizate:

Înainte de prezentarea algoritmilor, este important de menționat structurile de date folosite în cadrul aplicației. Pentru algoritmul Kruskal, muchiile grafului sunt reținute într-un **tablou de structuri de tip muchie**, fiecare conținând cele două extremități și costul asociat.

```
struct muchie
{
    int i, j, cost;
};

muchie x[5000];           // vectorul muchiilor (Kruskal)
int cost[101][101];       // matricea costurilor (Prim)
int n, m;                 // noduri, muchii
```

Alegerea unui tablou static în locul unui vector din STL a fost deliberată, pentru a demonstra înțelegerea modului în care memoria este gestionată manual în C++.

În continuare, voi prezenta câteva detalii interesante despre fiecare algoritm.

Tehnica Union-Find în Kruskal:

Cea mai importantă parte a algoritmului Kruskal este **evitarea ciclurilor**. Aceasta se realizează prin tehnica **Union-Find**: fiecare nod are un reprezentant (rep[i]), iar două noduri aparțin aceluiași subarbor dacă și numai dacă au același reprezentant. La adăugarea unei muchii, cei doi subarbori se reunesc prin actualizarea reprezentanților:

```
for (int i = 0; i < m && cnt < n - 1; i++) // APM are exact n-1 muchii
{
    int ri = rep[x[i].i];
    int rj = rep[x[i].j];

    if (ri != rj) // extremitatile sunt in subarbori diferiti
    {
        cnt++;
        v[i] = 1;
        S += x[i].cost;

        // reunirea celor doi subarbori
        for (int j = 1; j <= n; j++)
            if (rep[j] == rj)
                rep[j] = ri;
    }
}
```

Verificarea conexității grafului (Kruskal):

După finalizarea algoritmului, se verifică dacă numărul de muchii selectate este exact $n-1$. Dacă nu, înseamnă că graful are cel puțin două componente conexe separate și un APM complet nu poate exista:

```
if (cnt < n - 1) // dacă graful nu e conex, Kruskal va găsi o pădure parțială minimă (nu un arbore)
{
    cout << "\n ATENTIE: Graful nu este conex! APM nu exista.\n";
    return;
}
```

Această verificare este importantă deoarece algoritmul Kruskal va găsi o **pădure parțială minimă** în cazul unui graf neconex, nu un arbore.

Inițializarea pentru Prim:

În loc de o constantă matematică infinit, se folosește valoarea 999999 ca limită superioară practică pentru costurile muchiilor, știind că inputul nu va depăși această valoare.

Nodul de start primește costul 0 tocmai pentru a fi primul selectat în prima iterație a algoritmului.

```
for (int i = 1; i <= n; i++)
{
    inAPM[i] = 0;
    minCost[i] = 999999; // "infinit"
    parinte[i] = -1;
}
minCost[1] = 0; // pornim din nodul 1
```

Selectarea nodului minim în Prim:

La fiecare pas, algoritmul caută manual nodul nevizitat cu costul minim — o operație $O(n)$ care, deși nu este cea mai eficientă variantă posibilă (o coadă de priorități ar fi mai rapidă), este transparentă și ușor de urmărit:

```
// alegem nodul nevizitat cu costul minim
int u = -1;
for (int i = 1; i <= n; i++)
    if (!inAPM[i] && (u == -1 || minCost[i] < minCost[u]))
        u = i;
```

Condiția $u == -1$ tratează cazul primei iterații, când niciun nod nu a fost selectat încă.

Actualizarea costurilor minime în Prim:

În algoritmul Prim, după ce un nod u este adăugat în APM, **costurile minime ale vecinilor săi nevizitați sunt actualizate**. Aceasta este operația centrală care face ca algoritmul să extindă întotdeauna APM-ul cu muchia de cost minim disponibilă:

```
// actualizam vecinii lui u
for (int v = 1; v <= n; v++)
    if (cost[u][v] != 0 && !inAPM[v] && cost[u][v] <= minCost[v])
    {
        minCost[v] = cost[u][v];
        parinte[v] = u;
    }
```

Tabloul **parinte[]** reține pentru fiecare nod din care nod a fost el adăugat în APM, permițând reconstituirea și afișarea muchiilor la final.

Verificarea conexității grafului (Prim):

În timpul execuției, dacă algoritmul încearcă să selecteze un nod al cărui **minCost** a rămas **999999** (infinit), înseamnă că acel nod nu are nicio muchie spre componenta deja construită. Programul detectează această situație și afișează **același mesaj de avertizare**.

```
// daca costul minim e tot "infinit", graful e neconex
if (minCost[u] == 999999 && pas > 0)
{
    cout << "\n ATENTIE: Graful nu este conex! APM nu exista.\n";
    return;
}
```

Meniul principal cu buclă do-while alternativă:

Bucla *while*($T \neq 0$) din meniul principal permite rularea repetată a algoritmilor fără a reporni programul, util pentru testarea mai multor grafuri consecutiv:

```
while (T != 0)
{
    cout << "\n===== \n";
    cout << "  ALGORITMI - ARBORE PARTIAL MINIM \n";
    cout << "===== \n";
    cout << "  1. Algoritmul lui Kruskal \n";
    cout << "  2. Algoritmul lui Prim \n";
    cout << "----- \n";
    cout << " Alegeti algoritmul: ";
    cin >> USER;

    if (USER == 1)
    {
        cout << "\n [Kruskal] \n";
        alg_kruskal();
    }
    else if (USER == 2)
    {
        cout << "\n [Prim] \n";
        alg_prim();
    }
    else
    {
        cout << " Optiune invalida! \n";
    }

    cout << "\n Doriti sa continuati? (1 = Da / 0 = Nu): ";
    cin >> T;
}

cout << "\n La revedere! \n";
```

Bibliografie

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C. – Introducere în algoritmi, Ed. a 3-a, MIT Press, 2009.
- [2] GeeksforGeeks – Kruskal's Minimum Spanning Tree Algorithm: <https://www.geeksforgeeks.org/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/>
- [3] GeeksforGeeks – Prim's Minimum Spanning Tree: <https://www.geeksforgeeks.org/prims-minimum-spanning-tree-mst-greedy-algo-5/>
- [4] Kruskal, J. B. – On the shortest spanning subtree of a graph and the traveling salesman problem, Proceedings of the American Mathematical Society, 1956.
- [5] pbinfo.ro – Algoritmul lui Kruskal: <https://www.pbinfo.ro/articole/19048/algoritmul-lui-kruskal>
- [6] pbinfo.ro – Algoritmul lui Prim: <https://www.pbinfo.ro/articole/2443/algoritmul-lui-prim>
- [7] Prim, R. C. – Shortest connection networks and some generalizations, Bell System Technical Journal, 1957.
- [8] Wikipedia – Kruskal's algorithm: https://en.wikipedia.org/wiki/Kruskal%27s_algorithm
- [9] Wikipedia – Prim's algorithm: https://en.wikipedia.org/wiki/Prim%27s_algorithm
- [10] Wikipedia – Minimum spanning tree: https://en.wikipedia.org/wiki/Minimum_spanning_tree